# std::thread Basics - Creating and Managing Threads

## Theory, Edge Cases & Code Examples

- ✓ std::thread Overview
- ✓ What is a Thread?
- ✓ Thread Lifecycle and Ownership
- ✓ Thread States
- ✓ Why std::thread Matters in Real-World Systems

→ *Swipe to learn* →

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 1. std::thread Overview

**Definition:**

- Fundamental building block for C++11 multithreading
- Portable, high-level interface for concurrent execution
- Defined in `<thread>` header
- Thread starts **immediately** upon construction

**Key Characteristics:**

| Property | Details |
|---|---|
| **Execution Start** | Immediate (upon object construction) |
| **Callable Types** | Function pointer, lambda, functor, member function |
| **Argument Passing** | Supports multiple arguments via perfect forwarding |
| **Platform Abstraction** | Wraps POSIX threads (Linux) or Win32 threads (Windows) |
| **Move Semantics** | Move-only (no copying allowed) |

**Basic Usage:**

```
#include <thread>

void task() {
    // Work happens here
}

int main() {
    std::thread t(task);  // Thread starts immediately
    t.join();             // Wait for completion
}
```

---

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 2. What is a Thread?

**Core Concept:**
- **Smallest unit of execution** within a process
- Shares memory space with other threads in same process
- Enables concurrent (parallel) execution

**Thread vs Process:**

| Aspect | Thread | Process |
|---|---|---|
| **Memory** | Shared within process | Isolated memory space |
| **Creation Cost** | Low (1-10ms) | High (10-100ms+) |
| **Communication** | Direct (shared memory) | IPC needed (pipes, sockets) |
| **Synchronization** | Required (mutexes, locks) | Optional (separate address spaces) |
| **Crash Impact** | Can crash entire process | Isolated (doesn't affect others) |

**Platform Abstraction:**
- C++11 `std::thread` provides **unified interface** across platforms
- Linux: Uses `pthread_create`, `pthread_join` internally
- Windows: Uses `CreateThread`, `WaitForSingleObject` internally
- Portable code works everywhere without platform-specific changes

---

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 3. Thread Lifecycle and Ownership (1/2)

### Lifecycle Stages:

```
Construction → Joinable → Joined/Detached → Destroyed
     ↓            ↓             ↓               ↓
  Starts       Running      Completed      Cleanup
immediately
```

### Critical Rules:

| Rule | Requirement | Consequence if Violated |
|---|---|---|
| **Join or Detach** | Must call one before destruction | `std::terminate()` → Program aborts |
| **Single Join** | Can only join once | Undefined behavior |
| **Single Detach** | Can only detach once | Undefined behavior |
| **Explicit Ownership** | Thread object owns thread | Prevents resource leaks |

### Example:

```
void correct_lifecycle() {
    std::thread t([]{ /* work */ });

    // MUST choose one:
    t.join();    // Option 1: Wait for completion
    // OR
    t.detach();  // Option 2: Run independently

}  // ✅ Safe destruction

void dangerous_lifecycle() {
    std::thread t([]{ /* work */ });
    // ❌ Missing join/detach
}  // std::terminate() called → program aborts!
```

## 📌 3. Thread Lifecycle and Ownership (2/2)

**Why This Design?**

- **Prevents accidental bugs:** Forces explicit lifetime management
- **No implicit behavior:** Programmer must decide join vs detach
- **RAII principle:** Resource (thread) tied to object lifetime

---

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 4. Thread States

### Joinable State:

| State | Description | `joinable()` Returns | Operations Allowed |
|---|---|---|---|
| **Default-constructed** | No thread running | `false` | Assignment, destruction |
| **Active/Running** | Thread executing | `true` | `join()`, `detach()`, `move` |
| **Joined** | Thread completed | `false` | Destruction, assignment |
| **Detached** | Running independently | `false` | Destruction, assignment |
| **Moved-from** | Ownership transferred | `false` | Destruction, assignment |

### State Transitions:

```
std::thread t;                    // State: Not joinable
std::cout << t.joinable();        // Output: false

t = std::thread([]{ /* work */ }); // State: Joinable
std::cout << t.joinable();         // Output: true

t.join();                          // State: Not joinable
std::cout << t.joinable();         // Output: false
```

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 5. Why std::thread Matters in Real-World Systems (1/2)

**Autonomous Vehicle Example:**

A typical autonomous vehicle requires parallel processing of:

| Component | Thread Responsibility | Frequency |
|---|---|---|
| **Perception** | Process camera/LiDAR data | 10-30 Hz |
| **Localization** | GPS + IMU fusion | 100 Hz |
| **Planning** | Path planning algorithms | 10 Hz |
| **Control** | Steering/throttle commands | 50-100 Hz |
| **Safety Monitor** | Collision detection | 100 Hz |

**Why Multithreading:**
- **Parallelism:** Process multiple sensors simultaneously
- **Latency:** Meet strict timing deadlines (33ms for 30Hz camera)
- **Responsiveness:** Control loop runs while planning updates
- **Resource Utilization:** Use all CPU cores efficiently

**Critical Requirements:**
- **Deterministic timing:** Threads must complete within deadlines
- **No data races:** Shared state requires synchronization (mutexes)
- **Exception safety:** One thread crash shouldn't kill system
- **Resource management:** Proper join/detach to prevent leaks

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 5. Why std::thread Matters in Real-World Systems (2/2)

**Code Example:**

```
// Simplified autonomous vehicle threading
void perception_thread() {
    while (running) {
        auto frame = camera.capture();
        detect_objects(frame);  // 20ms budget
    }
}

void planning_thread() {
    while (running) {
        auto path = compute_path();
        publish(path);  // 100ms budget
    }
}

int main() {
    std::thread perception(perception_thread);
    std::thread planning(planning_thread);

    // Threads run in parallel

    perception.join();
    planning.join();
}
```

# std::thread Basics - Creating and Managing Threads

## Theory

## 📌 6. Quick Reference

### Creation Patterns:

| Pattern | Code | Use Case |
|---|---|---|
| Function | `std::thread t(func, args)` | Simple functions |
| Lambda | `std::thread t([]{...})` | Inline tasks |
| Member Function | `std::thread t(&Class::method, &obj)` | Object methods |

### Lifecycle Management:

| Operation | Code | Purpose |
|---|---|---|
| Join | `t.join()` | Wait for thread to finish |
| Detach | `t.detach()` | Let thread run independently |
| Check state | `t.joinable()` | Test if join/detach allowed |
| Move | `t2 = std::move(t1)` | Transfer ownership |

---

# std::thread Basics - Creating and Managing Threads

## Edge Cases

### 📌 1. Forgetting to Join or Detach

```
void dangerous_pattern() {
    std::thread t([]{
        std::cout << "Running in thread\n";
    });
    // ❌ t goes out of scope without join() or detach()
}  // std::terminate() called here - program aborts!
```

When a `std::thread` object with an associated thread of execution is destroyed (goes out of scope), its destructor checks if the thread is still joinable. If true, it calls `std::terminate()` to abort the program. This design prevents undefined behavior from threads outliving their controlling object.

**Why this design?** The C++ standard committee chose program termination over implicit join (blocking) or implicit detach (potential dangling references) to force explicit lifetime management.

**Solution:**

```
void safe_pattern() {
    std::thread t([]{
        std::cout << "Running in thread\n";
    });
    if (t.joinable()) {  // ✅ Always check before join/detach
        t.join();
    }
}
```

# std::thread Basics - Creating and Managing Threads

## Edge Cases

### 📌 2. Passing Arguments by Reference Without std::ref

```
void increment(int& x) {
    ++x;
}

void wrong_approach() {
    int counter = 0;
    std::thread t(increment, counter);  // ❌ Passed by VALUE, not reference!
    t.join();
    std::cout << counter << "\n";  // Still 0 - modification lost!
}

void correct_approach() {
    int counter = 0;
    std::thread t(increment, std::ref(counter));  // ✅ Use std::ref
    t.join();
    std::cout << counter << "\n";  // Now 1 - modification preserved
}
```

Thread arguments are **copied by default** into thread-local storage before the thread starts. This prevents dangling references if the parent scope ends before the thread completes. To pass by reference, you must explicitly use `std::ref()` or `std::cref()` (for const references).

**Internals:** The thread constructor uses perfect forwarding and stores decayed argument types. `std::ref()` creates a `std::reference_wrapper` that the thread unwraps to obtain the original reference.

# std::thread Basics - Creating and Managing Threads

## Edge Cases

### 📌 3. Double Join or Double Detach

```
std::thread t([]{ });
t.join();
t.join();  // ❌ Undefined behavior! Thread already joined.

std::thread t2([]{ });
t2.detach();
t2.detach();  // ❌ Undefined behavior! Thread already detached.
```

Once a thread is joined or detached, it becomes non-joinable (`joinable()` returns false). Attempting to join or detach again results in undefined behavior. The internal thread handle becomes invalid after the first operation.

**Best practice:** Always check `joinable()` before calling `join()` or `detach()`, especially in error-handling paths.

```
std::thread t([]{ });
// ... some operations that might throw ...
if (t.joinable()) {
    t.join();  // ✅ Safe
}
```

# std::thread Basics - Creating and Managing Threads

## Edge Cases

### 📌 4. Detached Threads Accessing Stack Variables

```
void dangerous_detach() {
    int local_data = 42;
    std::string message = "Hello";

    std::thread t([&local_data, &message]{
        std::this_thread::sleep_for(std::chrono::seconds(2));
        std::cout << local_data << " " << message << "\n";  // ❌ Dangling references!
    });

    t.detach();  // Thread continues after function returns
}  // local_data and message destroyed, but thread still accesses them!
```

Detached threads run independently and may outlive the scope that created them. Capturing stack variables by reference in detached threads is a common source of undefined behavior and crashes.

**Solution:** Pass by value or use heap allocation:

```
void safe_detach() {
    auto data = std::make_shared<int>(42);
    auto message = std::make_shared<std::string>("Hello");

    std::thread t([data, message]{  // ✅ Captured by value (shared_ptr copied)
        std::this_thread::sleep_for(std::chrono::seconds(2));
        std::cout << *data << " " << *message << "\n";
    });

    t.detach();  // Safe - shared_ptr keeps data alive
}
```

# std::thread Basics - Creating and Managing Threads

## Edge Cases

### 📌 5. Move-Only Nature of std::thread

```
std::thread t1([]{ });
std::thread t2 = t1;  // ❌ Compilation error - threads cannot be copied

std::thread t3 = std::move(t1);  // ✅ Ownership transferred
// t1 is now in "empty" state (not joinable)
// t3 owns the thread
```

`std::thread` is move-only (deleted copy constructor and copy assignment). This prevents multiple `std::thread` objects from owning the same thread of execution, which would create ambiguity about which destructor should join/detach.

---

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 1. Basic Thread Creation with Function Pointer

This demonstrates the simplest thread creation pattern. The thread begins execution immediately when constructed, and the main thread blocks at `join()` until the worker thread completes. This is the fundamental synchronization primitive for thread completion.

```
#include <iostream>
#include <thread>

void print_hello() {
    std::cout << "Hello from thread " << std::this_thread::get_id() << "\n";
}

int main() {
    std::thread t(print_hello);  // ✅ Thread starts immediately
    t.join();                     // Wait for thread to finish
    std::cout << "Main thread continues\n";
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 2. Thread with Function Arguments

Arguments are passed directly after the callable in the thread constructor. They are forwarded to the thread's execution context. Note that string literals are automatically converted to `std::string` during the forwarding process.

```
#include <iostream>
#include <thread>
#include <string>

void process_data(const std::string& sensor_name, int reading_count, double threshold) {
    std::cout << "Processing " << reading_count << " readings from " << sensor_name
              << " with threshold " << threshold << "\n";
}

int main() {
    std::thread lidar_thread(process_data, "LiDAR", 1000, 0.95);
    std::thread camera_thread(process_data, "Camera", 30, 0.85);

    lidar_thread.join();
    camera_thread.join();
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 3. Passing by Reference with std::ref

Using `std::ref()` for mutable references and `std::cref()` for const references ensures modifications in the thread affect the original variables. This is essential for returning results from thread computations.

```
#include <iostream>
#include <thread>

void compute_sum(const std::vector<int>& data, int& result) {
    result = 0;
    for (int val : data) {
        result += val;
    }
}

int main() {
    std::vector<int> numbers = {1, 2, 3, 4, 5};
    int sum = 0;

    std::thread t(compute_sum, std::cref(numbers), std::ref(sum));
    t.join();

    std::cout << "Sum: " << sum << "\n";  // Output: 15
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 4. Lambda Functions with Threads

Lambdas are the most common way to create threads for inline tasks. Capture by reference is safe when the thread is joined before the lambda's captured variables go out of scope.

```
#include <iostream>
#include <thread>

int main() {
    int counter = 0;

    std::thread t([&counter]{  // ✅ Capture by reference (safe because we join)
        for (int i = 0; i < 100; ++i) {
            ++counter;
        }
    });

    t.join();  // Must join before accessing counter
    std::cout << "Counter: " << counter << "\n";  // Output: 100
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 5. Member Function as Thread Entry Point

To call a member function in a thread, pass the member function pointer followed by the object pointer (or reference). The thread invokes the member function on the specified object.

```
#include <iostream>
#include <thread>

class SensorProcessor {
    std::string sensor_name_;
public:
    SensorProcessor(const std::string& name) : sensor_name_(name) {}

    void process() {
        std::cout << "Processing " << sensor_name_ << "\n";
    }
};

int main() {
    SensorProcessor camera("Camera");

    // Pass member function pointer and object reference
    std::thread t(&SensorProcessor::process, &camera);

    t.join();
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 6. Detached Thread for Background Task

Detached threads are useful for fire-and-forget background tasks. However, you must ensure the program doesn't exit before the detached thread completes (no automatic synchronization).

```
#include <iostream>
#include <thread>
#include <chrono>

void background_logger() {
    for (int i = 0; i < 5; ++i) {
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
        std::cout << "Log entry " << i << "\n";
    }
}

int main() {
    std::thread logger(background_logger);
    logger.detach();  // ✅ Thread runs independently

    std::cout << "Main continues immediately\n";

    // Must ensure main doesn't exit before detached thread completes
    std::this_thread::sleep_for(std::chrono::seconds(3));
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 7. Managing Multiple Threads with Container

Using `emplace_back()` constructs the thread directly in the container, avoiding unnecessary moves. This is the standard pattern for managing thread pools or multiple concurrent tasks.

```
#include <iostream>
#include <thread>
#include <vector>

void worker_task(int id) {
    std::cout << "Worker " << id << " executing\n";
}

int main() {
    std::vector<std::thread> workers;

    // Create 5 worker threads
    for (int i = 0; i < 5; ++i) {
        workers.emplace_back(worker_task, i);  // ✅ Construct in-place
    }

    // Join all threads
    for (auto& t : workers) {
        t.join();
    }
}
```

# std::thread Basics - Creating and Managing Threads

## Code Examples

### 📌 8. Getting Thread ID

Each thread has a unique ID of type `std::thread::id`, useful for debugging, logging, and thread-specific data structures. The ID is guaranteed unique during the thread's lifetime.

---

```
#include <iostream>
#include <thread>
#include <sstream>

void print_thread_info() {
    std::thread::id this_id = std::this_thread::get_id();
    std::ostringstream oss;
    oss << this_id;
    std::cout << "Thread ID: " << oss.str() << "\n";
}

int main() {
    std::thread t1(print_thread_info);
    std::thread t2(print_thread_info);

    std::cout << "Main thread ID: " << std::this_thread::get_id() << "\n";

    t1.join();
    t2.join();
}
```

# What You Learned

## Key Takeaways

- ✓ std::thread Overview
- ✓ What is a Thread?
- ✓ Thread Lifecycle and Ownership
- ✓ Thread States

*→ Swipe to learn →*